University of Technology Sydney

FEIT — 32555 Fundamentals of Software Development

Assessment 1 – Part 1: Requirements Analysis

Project: Requirements Analysis and Software Design

Group: Cmp1-07

Group Members:

1. Mathis Arthur Beissner

2. Shivakumar Kothapally

3. Siddharth Shanbhag

4. Varun Pocham

1. Introduction

This report is on the Part 1 requirements analysis of CLIUniApp / GUIUniApp. It is a self-enrolment system that we are designing for a local university. There are two types of users. Students register, login, enroll in up to four subjects per semester, remove subjects, view their list of enrollments and change their password; Existing university staff, admins do not need to register, can see all students, can sort by grade or by PASS/FAIL, can remove a student or wipe the whole list. All of it is stored in one shared file, students.data and read and write from the side of the student and administrator

The rest of the report is divided into 3 sections. The first is the user-story backlog, where we broke the brief down into individual, numbered stories. The second is the UML use case diagram, which covers the actors and how their use cases connect. The third and final section is the UML class diagram, which is basically the blueprint which we'll build from in Part 2.

2. User-Story Backlog

The brief was broken down into 22 user stories, each given a unique three digit ID. The same feature gets the stories with the same first digit, so they are easy to distinguish: 100s are login/credentials (including a separate admin login), 200s are registration, 300s are enrollment, 400s are removing a subject, 500s are viewing enrollments, 600s are password changes, 700s are the admin operations, and 800s are the file save/load stories.

2.1 User Stories

100 – Log in to student subsystem with your registered credentials

101 - Log in to the student management subsystem with registered credentials

102 – Verify that the email and password combination belongs to a registered student

200 – Register a new student by submitting a name, university email and password

201 – Verify if given password matches the pre-defined university password pattern

202 – Check that provided email matches predefined university email pattern

203 – Give student unique student ID on successful registration

300 – Add new subject to student's enlistment list

301 – Verify that the student's subject enrollment is more than maximum of four subjects

302 – Assign autogenerated ID to the new subject enrollment

303 – Assign autogenerated mark to the new subject enrollment

304 – Calculate subject grade based on the subject mark

400 – Remove subject from student's enrollment list

500 – Show a student's current enrollment list

600 – Change a student's password

700 – Show all registered students to an administrator

701 – Show students sorted by grade

702 – Show students sorted as PASS or FAIL

703 – Remove a single student from the registered students list

704 – Clear complete registered students list

800 – Save registered student data to file

801 - Supply stored student and enrollment data to the system

2.2 Requirements Table

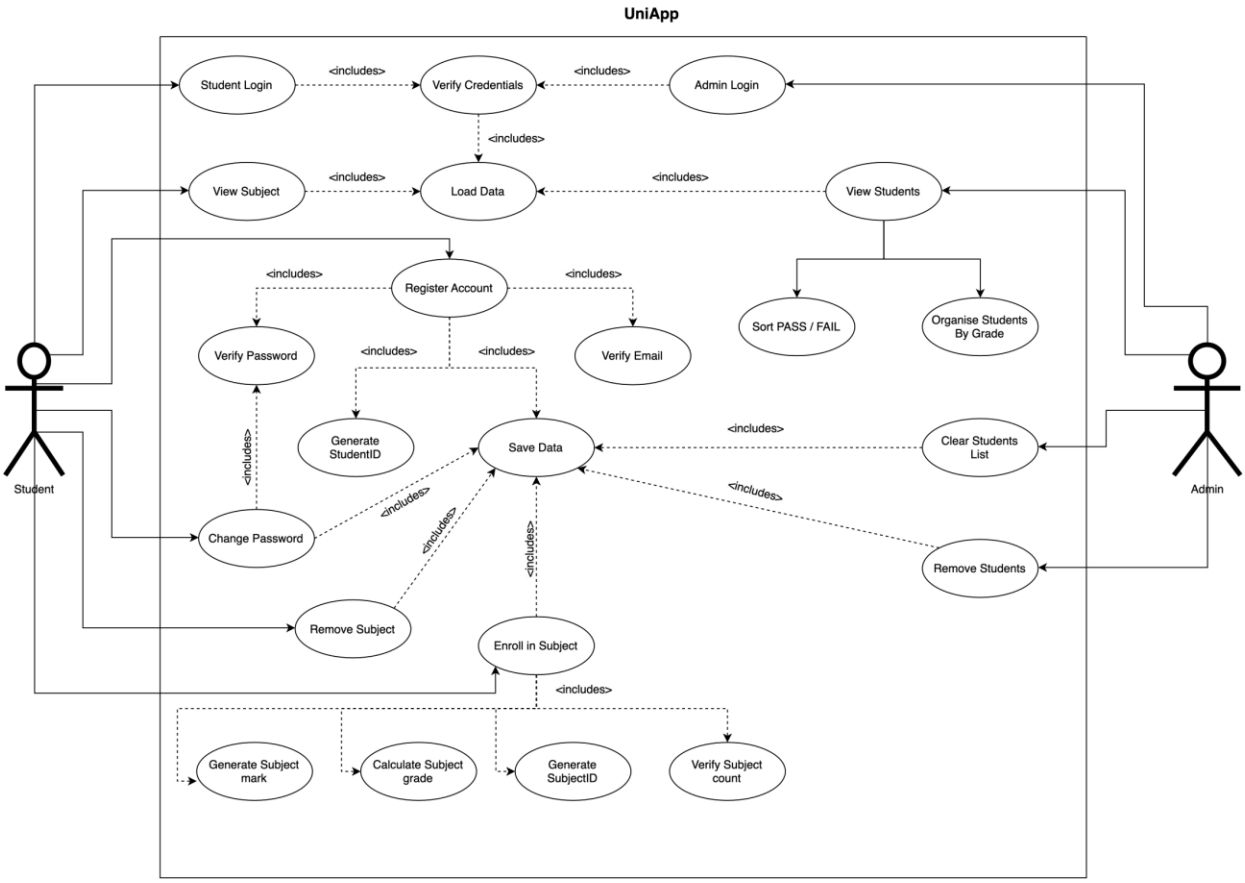
Each story is mapped below to who or what performs the action, what the action is, what the outcome is, and the function that will end up implementing it.

ID	User	Action	Result	Function
100	Student	Enter login credentials	If credentials are correct, student enters enrolment system; else error	studentLogin()
101	Admin	Enter login credentials	If credentials are correct, admin enters admin student management subsystem; else error	adminLogin()
102	System	Verify login credentials	Show error if invalid	verifyCredentials()

200	Student	Enter registration details: email, name & password	If details are unique and match the predefined patterns, an account is created; else error	registerAccount()
201	System	Verify password	Show error if invalid	verifyPassword()
202	System	Verify email address	Show error if invalid	verifyEmail()
203	System	Generate student ID	Unique 6-digit, zero-padded student ID added to student data	generateStudentID()
300	Student	Enrol in subject	New subject added to student's enrolment list	enrol()
301	System	Verify subject count	Show error if subject count exceeds 4	verifySubjectCount()
302	System	Generate subject ID	Unique, 3-digit zero-padded subject ID is assigned to subject	generateSubjectID()
303	System	Generate subject mark	Mark between 25 and 100 is assigned to subject	generateSubjectMark()
304	System	Calculate subject grade	Grade (Z, P, C, D, HD) is assigned to subject, based on subject mark	calculateSubjectGrade()
400	Student	Remove subject	Subject is removed from student's enrolment list	removeSubject()
500	Student	View enrolled subjects	Subject enrolment list is displayed	viewSubjects()
600	Student	Change password	If password matches predefined pattern, a new password is set; else error	changePassword()
700	Admin	View all students	Registered student list is displayed	viewStudents()
701	Admin	Organise students by grade	Grade-organised student list is displayed	sortByGrade()
702	Admin	Categorise students as PASS or FAIL	Categorised student list is displayed	sortPassFail()
703	Admin	Remove student	Selected student is removed from registered student list	removeStudent()
704	Admin	Clear student list	All students are removed from registered student list	clearStudentList()
800	System	Save student data	Student data is saved to students.data file	saveStudentData()
801	System	Load student data	Student data is loaded from students.data file	loadStudentData()

3. UML Use Case Diagram

Below is our use case diagram, with UniApp as the system boundary and two actors, Student and Admin, connected to the use cases they can trigger.



3.1 Actors

A Student is someone who’s registered. They can login, view their current enrollment, create a new account (obviously, before they've logged in), change their password, drop a subject, or enroll into a subject. These are the six use cases that are directly related to the Student actor in the diagram.

An Admin is current university staff, so they don't need to register at all. They login with their own Admin Login use case. From there they can view every student, clear the whole student list or remove one student.

3.2 Use Cases

You start with login . Student Login and Admin Login both have Verify Credentials , because verifying a username/password pair is the same logic either way , no need to build it twice . Verify Credentials then includes Load Data because you can't check anything against students.data in memory until it is actually read.

Register Account How a new student sets up an account. It pulls in three smaller use cases to do the actual checking — Verify Email, Verify Password, Generate StudentID — and also includes Save Data, since the whole point of registering is that it gets written to the file.

To enrolll in a subject is a little more complex. The first thing Subject checks is the Subject Count so that the system can enforce the four-subject limit before anything else can happen. It also has Generate SubjectID and that use case chains into Calculate Subject Grade, which itself includes Generate Subject Mark. That chain is the real order of things: give the subject an ID, generate a mark for it, then work out the grade from it. Save Data is included, as are all things that change data. Enrolll in Subject is no exception. Both Remove Subject and Change Password work the same way, both include Save Data. Change Password also includes Verify Password so a new password has to pass the same checks as one used at registration.

Load Data is the same as before, the data has to be loaded before it can be displayed. View Subject is the student-side use case for viewing your enrollment list.

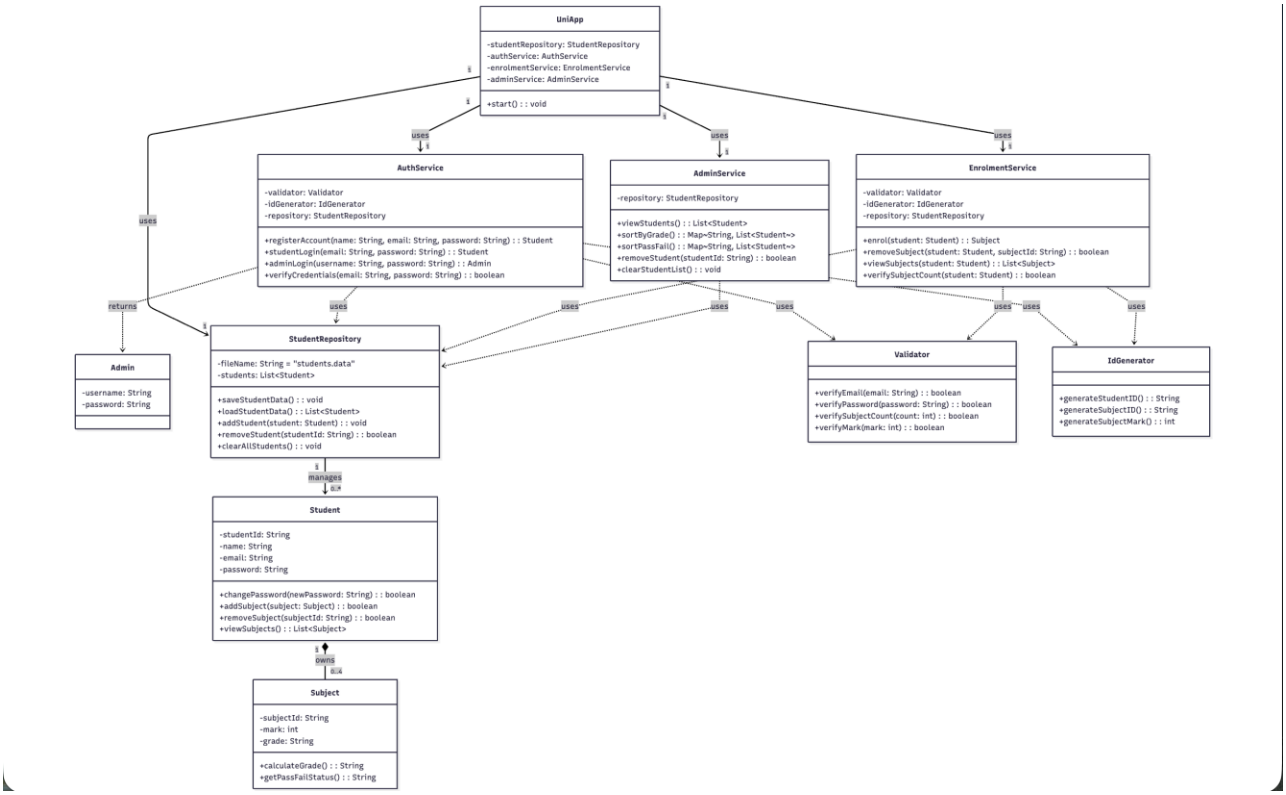
On the admin side, View Students has Load Data, then breaks off into Organize Students By Grade and Sort PASS/FAIL which just shows that same loaded list in two different ways. Since either operation permanently changes the contents of students.data, Save Data is included in both Clear Students List and Remove Students.

3.3 Relationships

This diagram contains only two types of relationships. Associations are represented by solid lines connecting an actor directly to the use cases it initiates. Dashed arrows labeled <<includes>> are include relationships, from a use case to another use case that must always be executed with it. We relied heavily on include relationships here, mainly to avoid repeating ourselves – Save Data, Load Data, and the validation use cases (Verify Password, Verify Email, Verify Subject Count) each only need to be drawn once, and then reused wherever Register Account, Enrolll in Subject, Remove Subject, Change Password, Clear Students List, or Remove Students needs them.

4. UML Class Diagram

Here's the class diagram. It splits the system into a UniApp controller, four service classes, one repository, two utility classes, and two classes that just hold data.



4.1 Classes and Attributes

UniApp sits on top and just starts everything (+start(): void) It privately references the four classes it relies on — -studentRepository, -authService, -enrolmentService and -adminService — and doesn't expose any of that directly, which is the point of keeping those fields private.

Our two data classes are Student and Subject. Student stores -studentId, -name, -email and -password, and has methods for the things a student can actually do to their own record: +changePassword(newPassword): boolean +addSubject(subject): boolean +removeSubject(subjectId): boolean +viewSubjects(): List<Subject> Subject stores only -subjectId, -mark and -grade, with +calculateGrade(): String and +getPassFailStatus(): String calculating the grade and pass / fail result from any mark stored.

We have three service classes: AuthService, EnrolmentService and AdminService. They don't really have any state of their own, other than a reference to the repository (AuthService and EnrolmentService also keep a Validator and an IdGenerator). The AuthService is responsible for +registerAccount(), +studentLogin(), +adminLogin() and +verifyCredentials(). +enrol(), +removeSubject(), +viewSubjects() and +verifySubjectCount(). EnrolmentService handles AdminService offers +viewStudents(), +sortByGrade(), +sortPassFail(), +removeStudent() and +clearStudentList().

File handling is all done in StudentRepository which stores -fileName (defaults to "students.data") and -students: List<Student> and provides the three services with the only way in and out of the file via +saveStudentData() +loadStudentData() +addStudent() +removeStudent() +clearAllStudents()

Validator and IdGenerator have no data whatsoever — they're just here so that the checking and generating logic is all in one place, instead of copy-pasted into both AuthService and EnrolmentService. Validator has +verifyEmail() +verifyPassword() +verifySubjectCount() +verifyMark() +generateStudentID() +generateSubjectID() +generateSubjectMark() IdGenerator

Admin is as simple as it can be - just -username and -password, no own methods. It is not directly created, when a login check is successful AuthService.adminLogin() returns it. So this is why the diagram now has a "returns" relationship from AuthService into Admin just like studentLogin() returns a Student.

4.2 Methods and Visibility

In the entire diagram, every attribute is private, which is just normal encapsulation – nothing outside the class has direct access to that data. All methods are public, because each is something that some other class *really* needs to be able to call. We didn't have any protected or package-private members, since all interaction between classes is done through the service layer anyway.

4.3 Relationships and Multiplicity

UniApp "uses" StudentRepository, AuthService, EnrolmentService and AdminService, it doesn't do any of the actual work itself, just hands everything off to other classes. AuthService and EnrolmentService both "use" Validator and IdGenerator and all three services "use" StudentRepository — so really just one shared repository, validator and ID generator instance that everything else points back to. AuthService also "returns" an Admin object, because a successful admin login needs to return an actual Admin instance for UniApp to work with, just as it returns a Student from studentLogin().

The last two relationships are structural, not "uses" dependencies. StudentRepository “manages” Student. 0..* on the Student side means it can hold any number of student records (including zero). Student “owns” Subject via composition (filled diamond), also 0..*, capped at four in practice by the enrollment rule - and since it is composition, a Subject cannot exist without its Student. Remove the student, and the subjects disappear with it.

5. Conclusion

If you put together the backlog, use case diagram and class diagram you can draw a straight line from what actually the university asked for to the design we are going to use for Part 2. The same shared behavior that we kept pulling out as include relationships in the use case diagram – validation, ID and mark generation, saving and loading – reappears as the Validator, IdGenerator and StudentRepository classes, so the Part 2 code should end up matching this analysis fairly closely.